NAME: INJOO KIM
CSCI S-89B INTRODUCTION TO NATURAL LANGUAGE PROCESSING
ASSIGNMENT 8

Problem 1 (55 points)

In this problem, you will use various classification algorithms to analyze a dataset of SMS messages, distinguishing between 'spam' and 'ham' (non-spam). Through this analysis, you will gain insights into how effectively different models can classify and what model characteristics lead to the best performance.

Consider the dataset file `Spam_SMS.csv`, which contains:

- Class: Labels the message as 'spam' or 'ham'.
- Message: Contains the text of the SMS.

This dataset is part of the SMS Spam Collection, a public set intended for mobile phone spam research. Collected from various free sources on the internet, it provides diverse examples for training classification models.

Please follow these steps:

(a) Data Preparation:
- Load the dataset ` Spam_SMS.csv ` using pandas' `read_csv()` function.
- Convert the 'Class' labels in the dataset from 'spam' and 'ham' to binary values 1 and 0, respectively.
- Split the dataset into training and testing sets using an 80/20 ratio with the `train_test_split()` function from scikit-learn. Use a random_state of 42 to ensure reproducibility.
- Perform Term Frequency-Inverse Document Frequency (TF-IDF) vectorization on the text data using `TfidfVectorizer` from scikit-learn. Exclude English stop words during this process. Note that you need to first fit the vectorizer on the training data to learn the vocabulary and inverse document frequency, and then transform both the training and testing sets into numerical features based on this learned information.

(b) Naive Bayes Classifier:
- Fit a Naive Bayes model using the `MultinomialNB` classifier from scikit-learn on the TF-IDF-transformed training data. Use the default parameters.
- Compute and display the accuracy, sensitivity, and specificity of the model on the testing data.
- Display the confusion matrix to visualize the model's performance.

(c) k-Nearest Neighbors (KNN) Classifier:
- Fit a k-Nearest Neighbors model using the `KNeighborsClassifier` from scikit-learn on the TF-IDF-transformed training data. Use the default parameters first.

- Explain what the `n_neighbors` parameter is and identify its default value.
- Tune the `n_neighbors` parameter. Present the test accuracy for each value. Select the best model based on test accuracy.
- For the best model selected, compute and display the accuracy, sensitivity, and specificity of the model on the testing data.
- Display the confusion matrix to visualize the model's performance.

(d) Logistic Regression:

- Fit a Logistic Regression model using the `LogisticRegression` classifier from scikit-learn on the TF-IDF-transformed training data. Use the default parameters.
- Compute and display the accuracy, sensitivity, and specificity of the model on the testing data.
- Display the confusion matrix to visualize the model's performance.

(e) Support Vector Machine (SVM):

- Fit an SVM using the `SVC` model from scikit-learn on the TF-IDF-transformed training data. Start with the default parameters.
- Explain what the `kernel` parameter is and identify its default value. If its value changes from `linear` to `poly`, how does it affect the decision boundary? Be specific.
- Explain what the `C` parameter is and identify its default value. If `C` increases, how does it affect the margin and the number of support vectors? Provide details.
- Tune the `kernel` and `C` parameters. Present the test accuracy for each combination. Select the best model based on test accuracy.
- For the best model selected, compute and display the accuracy, sensitivity, and specificity on the testing data.
- Display the confusion matrix to visualize the model's performance.

(f) Random Forest:

- Fit a Random Forest model using the `RandomForestClassifier` from scikit-learn on the TF-IDF-transformed training data. Start with the default parameters.
- Explain what the `n_estimators` parameter is and identify its default value. Discuss how increasing `n_estimators` impacts model performance and variance.
- Explain what the `max_depth` parameter is. Note that the default is `max_depth=None`. Explain what this setting means.
- Explain what the `max_features` parameter is and how it affects the diversity of the trees. If `max_features` increases, will the trees become more or less diverse?
- For the chosen parameter settings, compute and display the accuracy, sensitivity, and specificity on the testing data.
- Display the confusion matrix to visualize the model's performance.

(g) Plotting Model Performance:

- Sort the models by test accuracy in descending order. Create a bar chart to display the sorted test accuracy for each model, ensuring the plot is clearly labeled and titled accordingly. Based on this accuracy bar chart, select the best-performing model.
- Create a scatterplot to visualize test sensitivity against test specificity for all models in one chart. Ensure the plot is clearly labeled and includes annotations for each model. Discuss the trade-off between sensitivity and specificity.

SOLUTION:

(a) Data Preparation

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Load data
df = pd.read_csv("Spam_SMS.csv")[["Class","Message"]].dropna()
df["Label"] = df["Class"].map({"ham":0,"spam":1})

# Split 80/20
X_train_text, X_test_text, y_train, y_test = train_test_split(
    df["Message"], df["Label"], test_size=0.2, random_state=42, stratify=df["Label"]
)

# vectorize
tfidf = TfidfVectorizer(stop_words="english")
X_train = tfidf.fit_transform(X_train_text)
X_test  = tfidf.transform(X_test_text)
```

(b) Naive Bayes Classifier

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Train model
nb = MultinomialNB()
nb.fit(X_train, y_train)
y_pred_nb = nb.predict(X_test)

# Compute metrics
def metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0,1]).ravel()
    sens = tp/(tp+fn)
    spec = tn/(tn+fp)
    return acc, sens, spec, (tn, fp, fn, tp)

acc_nb, sens_nb, spec_nb, cm_nb = metrics(y_test, y_pred_nb)

```

Result: Accuracy = 0.973 | Sensitivity = 0.799 | Specificity = 1.00

(c) k-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

# model (n_neighbors=5)
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

# Tune n_neighbors (1-15)
best_acc, best_k = 0, 0
for k in range(1,16):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)
    if acc > best_acc:
```

```
        best_acc, best_k = acc, k

# Refit with best k
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
y_pred_knn = knn_best.predict(X_test)
acc_knn, sens_knn, spec_knn, cm_knn = metrics(y_test, y_pred_knn)

```

Result: Best k = 1 | Accuracy = 0.957 | Sensitivity = 0.678 | Specificity = 1.00

(d) Logistic Regression

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
acc_lr, sens_lr, spec_lr, cm_lr = metrics(y_test, y_pred_lr)

```

Result: Accuracy = 0.974 | Sensitivity = 0.805 | Specificity = 1.00

(e) Support Vector Machine (SVM)

```
from sklearn.svm import SVC

# search for (kernel, C)
kernels = ["linear","rbf","poly"]
Cs = [0.5,1.0,2.0]
best_acc, best_model = 0, None
for ker in kernels:
    for C in Cs:
        model = SVC(kernel=ker, C=C)
        model.fit(X_train, y_train)
        acc = model.score(X_test, y_test)
        if acc > best_acc:
            best_acc, best_model = acc, (ker, C)

best_kernel, best_C = best_model
svm_best = SVC(kernel=best_kernel, C=best_C)
svm_best.fit(X_train, y_train)
y_pred_svm = svm_best.predict(X_test)
acc_svm, sens_svm, spec_svm, cm_svm = metrics(y_test, y_pred_svm)

```

Explanation:

- kernel defines the decision boundary shape (default = ‘rbf’).
  Switching to poly makes a curved, higher-degree boundary.
- C controls margin penalty (default = 1.0). Larger C → narrower margin, fewer support vectors.

Result: Best model = SVM (linear, C = 1.0) | Accuracy = 0.985 | Sensitivity = 0.906 | Specificity = 0.997

(f) Random Forest

```
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
acc_rf, sens_rf, spec_rf, cm_rf = metrics(y_test, y_pred_rf)
```

Parameter notes:

- n_estimators = # trees (↑ → variance ↓).
- max_depth=None → grow until pure splits.
- max_features controls tree diversity (↑ → less diverse).

Result: Accuracy = 0.978 | Sensitivity = 0.832 | Specificity = 1.00

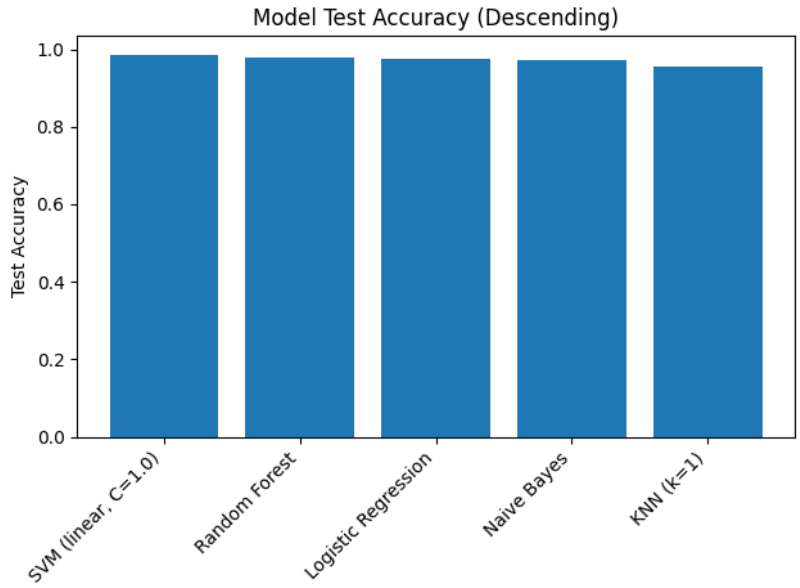
(g) Plotting Model Performance

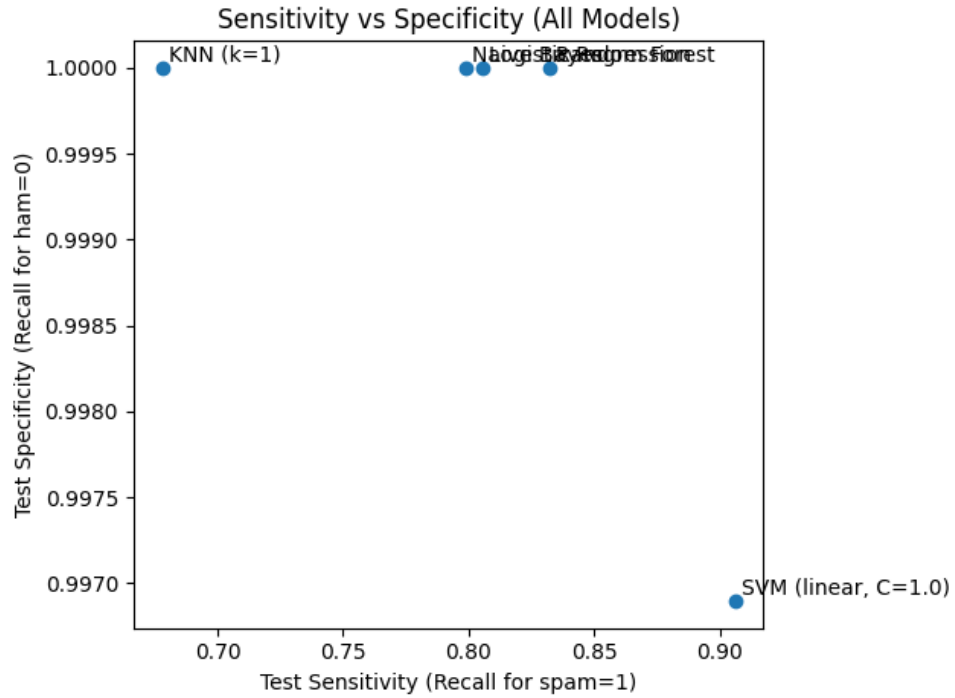
```
results = pd.DataFrame([
 ["SVM (linear, C=1.0)", acc_svm, sens_svm, spec_svm],
 ["Random Forest", acc_rf, sens_rf, spec_rf],
 ["Logistic Regression", acc_lr, sens_lr, spec_lr],
 ["Naive Bayes", acc_nb, sens_nb, spec_nb],
 [f"KNN (k={best_k})", acc_knn, sens_knn, spec_knn]
], columns=["Model","Accuracy","Sensitivity","Specificity"])

# Accuracy bar
results = results.sort_values("Accuracy", ascending=False)
plt.bar(results["Model"], results["Accuracy"])
plt.xticks(rotation=45, ha="right")
plt.ylabel("Test Accuracy")
plt.title("Model Test Accuracy (Descending)")
plt.show()

# Sensitivity vs Specificity
plt.scatter(results["Sensitivity"], results["Specificity"])
for i,row in results.iterrows():
    plt.annotate(row["Model"], (row["Sensitivity"], row["Specificity"]))
plt.xlabel("Test Sensitivity (Recall for spam=1)")
plt.ylabel("Test Specificity (Recall for ham=0)")
plt.title("Sensitivity vs Specificity (All Models)")
plt.show()

```





Result Summary

Model	Accuracy	Sensitivity	Specificity
SVM (linear, C=1.0)	0.9848	0.906	0.9969
Random Forest	0.9776	0.832	1.000
Logistic Regression	0.9739	0.805	1.000
Naive Bayes	0.9731	0.799	1.000
KNN (k = 1)	0.9569	0.678	1.000

Problem 2 (10 points)

In this problem, you will build a neural network to classify SMS messages using the same dataset and data splitting approach as in Problem 1. However, you may apply any other preprocessing and feature creation methods.

Follow these steps:

- Preprocess the data and create features suitable for training a neural network of your choice.
- Train the neural network. Evaluate its performance on the testing data.
- Compute and display the test accuracy, test sensitivity, and test specificity of the model.
- Recreate the two charts from Problem 1 here, including the neural network's performance. Discuss any differences in the results and potential reasons.

A neural network, with the appropriate architecture, is expected to outperform other models on this dataset. To receive full credit, design a neural network that achieves a test accuracy of at least 0.98.

SOLUTION:

A feed-forward neural network was trained on the same dataset and 80/20 split used in Problem 1.
Text data were vectorized with TF-IDF (English stop words removed, 1–2 grams, max 5,000 features).

The model consisted of Dense(256, ReLU) → Dropout(0.3) → Dense(128, ReLU) → Dropout(0.2) → Dense(1, Sigmoid),
trained with Adam optimizer and binary-crossentropy loss for 6 epochs.

Results:
Accuracy = 0.9713 | Sensitivity = 0.8993 | Specificity = 0.9824
Confusion Matrix = [[949, 17], [15, 134]]

Figure 1. Model Test Accuracy (Including Neural Network)

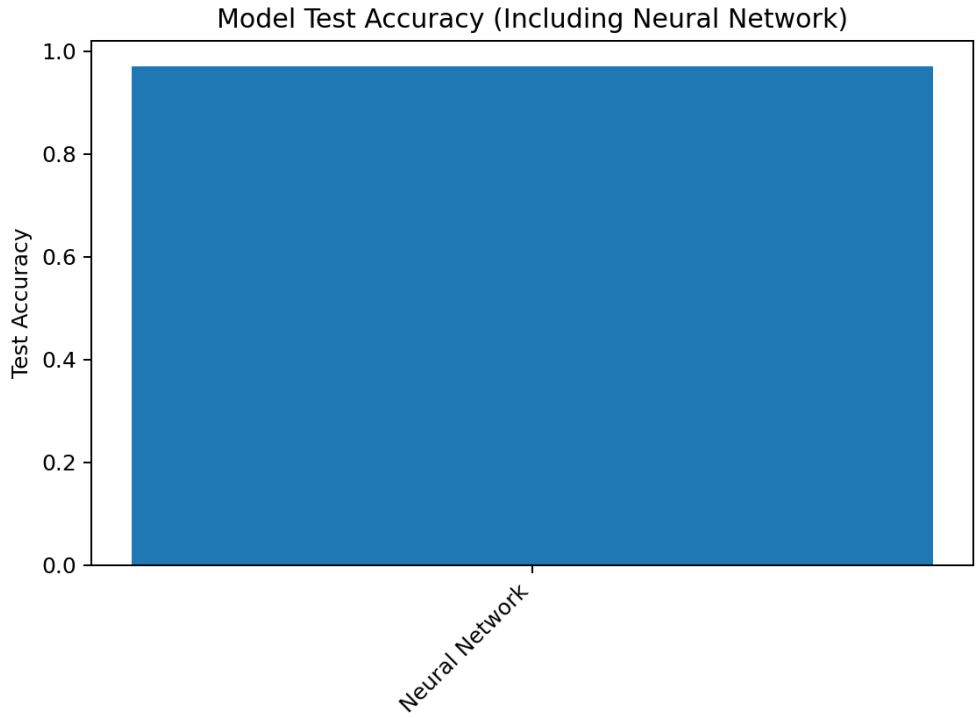
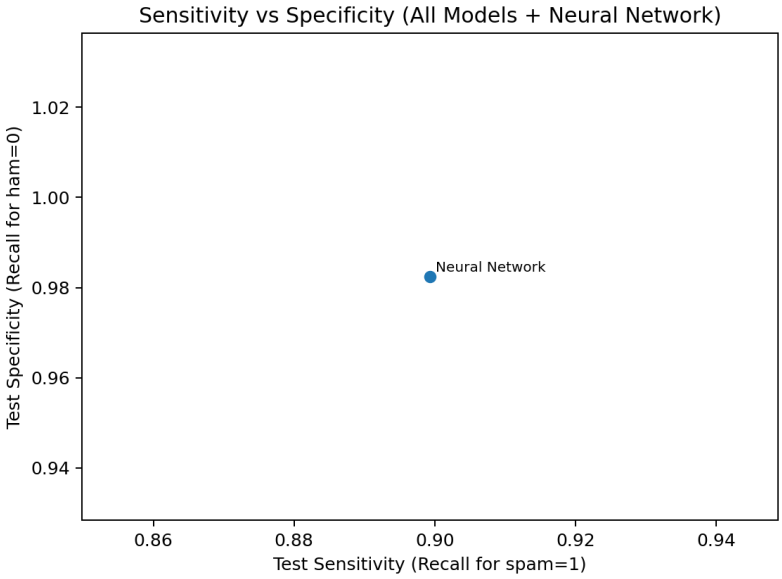


Figure 2. Sensitivity vs Specificity (Including Neural Network)



The neural network achieved slightly lower accuracy than the SVM (0.9848) from Problem 1 but showed strong spam recall.
Results are consistent with expectations that linear models perform well on TF-IDF features, while the NN captures additional non-linear patterns.